

Relatório Técnico — Trabalho da Unidade I

Disciplina: DIM0501 — Boas Práticas de Programação

Tema: Aplicação dos conceitos de boas práticas em cenário simulado

Semestre: 2026.1

1. Identificação do Grupo

- **Integrante 1:** Luan Pedro Abreu Vieira
- **Linguagem utilizada:** Python
- **Link do repositório Git:** <https://github.com/LuanPedro01/trabalho-bpp-unidade-1>

2. Descrição do Sistema

O sistema é um **Monitor de Fake News e Qualidade da Informação**: um pequeno aplicativo de console que permite cadastrar textos de notícias e classificá-los como **confiável**, **duvidosa** ou **falsa**. A classificação pode ser informada manualmente pelo usuário ou determinada automaticamente por uma heurística que considera:

- **Ausência de fonte declarada** no texto;
- **Linguagem sensacionalista** (uso de "!!!" ou "URGENTE");
- **Texto muito curto** (menos de 10 caracteres).

O objetivo do código original é servir de exemplo simples de coleta e classificação de informações, podendo ser usado em contextos didáticos ou como base para sistemas mais robustos de checagem (ex.: triagem inicial de notícias antes de uma verificação humana).

3. Problemas Identificados

3.1 Problemas de Legibilidade

- Nomes de variáveis e funções pouco descritivos: `data`, `f`, `func2`, `t`, `c`, `d`, `b`, `op`, `txt`, `score`.
- Comentários redundantes e sem valor (apenas reescrevem o código).
- Ausência total de docstrings e anotações de tipo.
- Uso de `score = score + 1` em vez do idiomático `score += 1`.
- Comparação `b == None` em vez de `b is None`.
- Iteração com índices (`range(0, len(data))`) em vez de iterar diretamente sobre a coleção.

Trecho de código (exemplo):

```

data = []

# função que faz tudo
def f(a, b=None):
    # essa função adiciona uma coisa
    if a != "":
        d = {}
        d["t"] = a
        if b == None:
            d["c"] = "duvidosa"
        else:
            d["c"] = b
        data.append(d)
    else:
        print("erro")

```

O nome `f` não revela intenção, os parâmetros `a` e `b` exigem leitura do corpo para serem entendidos, e o comentário “função que faz tudo” é, ao mesmo tempo, desnecessário e indicador de **má modelagem** (a função realmente faz coisas demais).

3.2 Problemas de Organização

- Todo o sistema fica em **um único arquivo** (`Sistema.py`), misturando:
 - estrutura de dados (`data` global);
 - regras de classificação (`analisar`);
 - interface com o usuário (`add_manual`, `add_auto`, `menu`).
- Uso de **estado global** (`data`) que pode ser modificado por qualquer parte do código.
- Chamada `menu()` direta no fim do módulo, em vez de `if __name__ == "__main__":`, o que impede importar o módulo sem executá-lo.
- A função `f` mistura **três responsabilidades**: validar o texto, montar a estrutura da notícia e adicioná-la ao armazenamento.

3.3 Código Duplicado ou Design Ruim

- `add_manual` e `add_auto` repetem o mesmo padrão de leitura do texto.
- Notícias são representadas por **dicionários soltos** (`{"t": ..., "c": ...}`) com chaves que ninguém valida nem documenta.
- **Strings mágicas** espalhadas: `"confiavel"`, `"duvidosa"`, `"falsa"`, `"FONTE"`, `"!!!"`, `"URGENTE"`.
- **Números mágicos**: `10` (tamanho mínimo do texto), `0` e `1` (limites de pontuação).
- A função `f` é um *flag argument* (recebe `b=None` para alternar entre dois comportamentos), o que dificulta a leitura.

```
def analisar(txt):
    score = 0
    if "FONTE" not in txt: score = score + 1
    if "!!!" in txt: score = score + 1
    if "URGENTE" in txt: score = score + 1
    if len(txt) < 10: score = score + 1
    if score == 0: return "confiavel"
    elif score == 1: return "duvidosa"
    else: return "falsa"
```

3.4 Falta de Validação

- `add_manual` aceita **qualquer string** como classificação, mesmo que não corresponda a `confiavel` / `duvidosa` / `falsa`.
- Texto contendo apenas espaços (" ") é considerado válido (a comparação é feita com `!= ""`).
- Não há tratamento para `EOFError` ou `KeyboardInterrupt` (Ctrl+C / Ctrl+Z).
- Mensagens de erro genéricas ("erro", "errado") que não orientam o usuário sobre o que houve.

3.5 Problemas de Documentação

- Comentários redundantes (`# lista tudo`, `# analisa o texto`, `# função que faz tudo`) que não acrescentam informação.
- Nenhum docstring em módulo, função ou classe.
- Ausência de **README** explicando como executar e quais são as regras de classificação.
- Critérios de classificação ficam implícitos em condicionais, em vez de estarem documentados.

4. Estratégias de Solução

4.1 Refatoração

- Renomeação de identificadores para refletir o domínio:
 - `data` → `RepositorioNoticias._noticias`
 - `f` → `RepositorioNoticias.adicionar` + construção via `Noticia(...)`
 - `func2` → `listar`
 - `analisar` → `classificar_noticia`
 - `add_manual` / `add_auto` → `adicionar_manualmente` / `adicionar_automaticamente`
 - `t`, `c`, `txt`, `score`, `op` → `texto`, `classificacao`, `indicios`, `opcao`.
- Substituição de **strings e números mágicos** por constantes (`PALAVRA_FONTE`, `MARCADOR_EXCLAMACAO`, `MARCADOR_URGENCIA`, `TAMANHO_MINIMO_TEXTO`, `LIMITE_CONFIAVEL`, `LIMITE_DUVIDOSA`).
- Quebra de funções grandes: a antiga `f` foi dividida em criação da entidade (`Noticia`) e armazenamento (`RepositorioNoticias.adicionar`).
- Uso de `Enum` (`Classificacao`) para garantir que apenas valores válidos circulem pelo sistema.

- Uso de `@dataclass(frozen=True)` para a `Noticia`, tornando-a imutável e dispensando `__init__` / `__repr__` manuais.
- Substituição de `range(0, len(data))` por iteração direta.
- `is None` no lugar de `== None`; `+=` no lugar de `x = x + 1`.

4.2 Modularização

A organização passou de **um único arquivo** para uma divisão em camadas:

- `modelo/` — entidades (`Noticia`, `Classificacao`).
- `servico/` — lógica (`classificar_noticia`, `RepositorioNoticias`).
- `interface/` — CLI (`executar`, ações do menu).
- `main.py` — apenas inicializa o programa.

A direção das dependências é unidirecional (`interface` → `servico` → `modelo`), o que permite, por exemplo, substituir a CLI por outra interface sem tocar nas regras de negócio.

4.3 Programação Defensiva

- **Validação do texto:** leitura através de `_ler_texto_obrigatorio`, que rejeita strings vazias ou apenas com espaços e pede nova entrada.
- **Validação da classificação:** `Classificacao.a_partir_de_texto` aceita variações de caixa/espaços, mas levanta `ValueError` com mensagem clara para valores inválidos.
- **Validação no analisador:** `classificar_noticia` levanta `ValueError` para texto vazio, em vez de retornar uma classificação arbitrária.
- **Validação no repositório:** `adicionar` exige uma instância de `Noticia`, levantando `TypeError` em caso contrário.
- **Tratamento de erros na CLI:** `try/except ValueError` no loop principal evita que entradas inválidas derrubem o programa, e `EEOFError` / `KeyboardInterrupt` são capturados para encerrar com mensagem amigável.
- **Mensagens descritivas** substituem `"erro"` e `"errado"` por textos que indicam o que ocorreu e como corrigir.

4.4 Documentação

- **Docstrings** em todos os módulos, classes e funções públicas, descrevendo propósito, argumentos, retorno e exceções relevantes.
- Comentários inúteis foram **removidos**; comentários que permanecem explicam decisões não-óbvias (ex.: por que `MARCADOR_EXCLAMACAO` e `MARCADOR_URGENCIA` são tratados como dois indícios independentes, preservando o comportamento original).
- **README.md** com:
 - descrição do sistema;
 - requisitos e instruções de execução;
 - explicação das funcionalidades e critérios de classificação;
 - estrutura de pastas e arquitetura;

- lista de boas práticas aplicadas.

5. Exemplos de Melhoria (Antes vs Depois)

Exemplo 1 — Função genérica `f` vira entidade + repositório

Antes:

```
data = []

def f(a, b=None):
    if a != "":
        d = {}
        d["t"] = a
        if b == None:
            d["c"] = "duvidosa"
        else:
            d["c"] = b
        data.append(d)
    else:
        print("erro")
```

Depois:

```
# modelo/noticia.py
class Classificacao(Enum):
    CONFIAVEL = "confiavel"
    DUVIDOSA = "duvidosa"
    FALSA = "falsa"

@dataclass(frozen=True)
class Noticia:
    texto: str
    classificacao: Classificacao

# servico/repositorio.py
class RepositorioNoticias:
    def __init__(self) -> None:
        self._noticias: List[Noticia] = []

    def adicionar(self, noticia: Noticia) -> None:
        if not isinstance(noticia, Noticia):
            raise TypeError("Apenas instâncias de Noticia podem ser adicionadas.")
        self._noticias.append(noticia)
```

Explicação: a função original `f` violava o **Princípio da Responsabilidade Única**: validava o texto, decidia o valor padrão da classificação, montava o dicionário e o adicionava em uma lista global. Na versão refatorada:

- a **estrutura da notícia** virou uma `dataclass` imutável e tipada;
- as **classificações válidas** ficam protegidas pelo `Enum`, eliminando strings mágicas;
- o **armazenamento** é encapsulado por `RepositorioNoticias`, que esconde a coleção interna e valida o tipo do objeto recebido;
- a validação do texto sai dessa camada e passa a ser feita na **interface** e no **analizador**, onde realmente faz sentido.

Exemplo 2 — Menu com `if/elif` e mensagens vagas

Antes:

```
def menu():
    while True:
        print("1 - adicionar manual")
        print("2 - adicionar automatico")
        print("3 - listar")
        print("4 - sair")

        op = input("opcao: ")

        if op == "1":
            add_manual()
        elif op == "2":
            add_auto()
        elif op == "3":
            func2()
        elif op == "4":
            break
        else:
            print("errado")

menu()
```

Depois:

```
# interface/cli.py
ACOES_MENU: Dict[str, AcaoMenu] = {
    OPCA0_ADICIONAR_MANUAL: adicionar_manualmente,
    OPCA0_ADICIONAR_AUTOMATICO: adicionar_automaticamente,
    OPCA0_LISTAR: listar,
}

def executar() -> None:
    repositorio = RepositorioNoticias()
    while True:
        _exibir_menu()
        try:
            opcao = input("opcao: ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break

        if opcao == OPCA0_SAIR:
            break

        acao = ACOES_MENU.get(opcao)
        if acao is None:
            print("Opção inválida. Escolha 1, 2, 3 ou 4.")
            continue

        try:
            acao(repositorio)
        except ValueError as erro:
            print(f"Erro: {erro}")
```

Explicação:

- A cadeia de `if/elif` foi substituída por um **mapa de opções** (`ACOES_MENU`), facilitando adicionar ou remover ações no futuro.
- As opções viraram **constantes nomeadas** (`OPCA0_LISTAR`, etc.), o que evita números mágicos e torna o código autoexplicativo.
- A mensagem "errado" foi substituída por uma orientação clara ("Opção inválida. Escolha 1, 2, 3 ou 4.").
- O programa passou a tratar `EOFError` e `KeyboardInterrupt`, encerrando de forma controlada em vez de exibir um *traceback*.
- Erros de validação levantados pelas ações (texto vazio, classificação inválida) são capturados em um único ponto, deixando cada ação focada em seu próprio fluxo principal.
- A chamada direta `menu()` no fim do módulo deu lugar a `main.py` com `if __name__ == "__main__"`, permitindo importar o pacote sem executar o loop.

Exemplo 3 — Função `analisar` com strings/números mágicos

Antes:


```
def analisar(txt):  
    score = 0  
    if "FONTE" not in txt:  
        score = score + 1  
    if "!!!" in txt:  
        score = score + 1  
    if "URGENTE" in txt:  
        score = score + 1  
    if len(txt) < 10:  
        score = score + 1  
  
    if score == 0:  
        return "confiavel"  
    elif score == 1:  
        return "duvidosa"  
    else:  
        return "falsa"
```

Depois:

```

# servico/analizador.py
PALAVRA_FONTE      = "FONTE"
MARCADOR_EXCLAMACAO = "!!!"
MARCADOR_URGENCIA  = "URGENTE"
TAMANHO_MINIMO_TEXTO = 10
LIMITE_CONFIAVEL    = 0
LIMITE_DUVIDOSA     = 1

def classificar_noticia(texto: str) -> Classificacao:
    if texto is None or not texto.strip():
        raise ValueError("Texto da notícia não pode ser vazio.")

    indicios = _contar_indicios(texto)

    if indicios <= LIMITE_CONFIAVEL:
        return Classificacao.CONFIAVEL
    if indicios <= LIMITE_DUVIDOSA:
        return Classificacao.DUVIDOSA
    return Classificacao.FALSA

def _contar_indicios(texto: str) -> int:
    indicios = 0
    if PALAVRA_FONTE not in texto:
        indicios += 1
    if MARCADOR_EXCLAMACAO in texto:
        indicios += 1
    if MARCADOR_URGENCIA in texto:
        indicios += 1
    if len(texto) < TAMANHO_MINIMO_TEXTO:
        indicios += 1
    return indicios

```

Explicação: a heurística passou a ser auto-documentada por meio de constantes com nomes descritivos. A função foi quebrada em duas: `classificar_noticia` valida a entrada e decide a classificação a partir da pontuação; `_contar_indicios` (privada) concentra os critérios. O retorno deixou de ser uma string solta e passou a ser um `Enum`, eliminando a possibilidade de typos como `"confiável"` (com acento) ou `"Confiavel"` se propagarem pelo sistema.

6. Organização Final do Código

```
projeto/
├── modelo/
│   ├── __init__.py
│   └── noticia.py           # Noticia (dataclass) e Classificacao (Enum)
├── servico/
│   ├── __init__.py
│   ├── analisador.py       # classificar_noticia e constantes do critério
│   └── repositorio.py      # RepositorioNoticias (armazenamento em memória)
├── interface/
│   ├── __init__.py
│   └── cli.py              # Menu e ações do usuário (CLI)
├── original/
│   └── Sistema.py          # Código inicial preservado para referência
├── main.py                 # Ponto de entrada
├── README.md
└── RELATORIO_TECNICO.md
```

Papel de cada parte:

- **modelo/** : define **o que** é uma notícia. Não conhece a heurística de classificação nem como ela é exibida ao usuário.
- **servico/** : contém as **regras de negócio** — o analisador, que decide a classificação a partir do texto, e o repositório, que guarda as notícias na sessão atual.
- **interface/** : encapsula a **interação com o usuário** — leitura do teclado, exibição do menu e formatação da saída.
- **main.py** : faz apenas a *bootstrap* da aplicação, chamando `interface.cli.executar()` quando executado como script.
- **original/Sistema.py** : mantido para que seja possível comparar a versão refatorada com a original sem precisar consultar o GitHub.

7. Conclusão

O exercício deixou claro que **o tamanho** do código não é, sozinho, indicador de simplicidade: o `Sistema.py` original cabe em uma tela, mas concentra problemas que tornam a manutenção e a evolução arriscadas. Renomear identificadores, extrair constantes, separar responsabilidades e adicionar validações foram mudanças aparentemente modestas, mas que mudaram a forma de conversar com o código — agora é possível **ler o nome de uma função e prever o que ela faz**, sem precisar mergulhar na sua implementação.

As maiores dificuldades foram:

1. **Decidir até onde refatorar** sem extrapolar para uma reescrita do zero. A diretriz "manter o funcionamento original" obrigou o grupo a preservar particularidades (como a contagem

separada de "!!!" e "URGENTE" ou a sensibilidade a maiúsculas em "FONTE"), mesmo sabendo que poderiam ser melhoradas.

2. **Definir a fronteira entre as camadas:** por exemplo, onde colocar a validação do texto vazio. Optamos por validar nas **bordas** — na entrada do CLI e na entrada do analisador — em vez de espalhar `if` s por todo o código.
3. **Equilibrar comentários e clareza:** vários comentários do código original foram retirados porque os nomes refatorados já contam a história; outros foram acrescentados apenas onde havia uma decisão não-óbvia (como a preservação do critério duplicado de sensacionalismo).

As boas práticas impactaram o código em três frentes principais:

- **Legibilidade:** ficou possível entender cada arquivo isoladamente, sem precisar reconstruir o contexto inteiro do programa.
- **Robustez:** o sistema deixou de quebrar diante de entradas inesperadas (Ctrl+C, classificações inválidas, texto vazio) e passou a oferecer mensagens informativas.
- **Evolução:** a separação em camadas e o uso de `Enum` / `dataclass` abrem caminho para novas funcionalidades (persistência em arquivo, novos critérios de análise, outra interface de usuário) sem precisar reabrir cada parte do sistema.

Em resumo, **o objetivo do trabalho — reduzir dívida técnica sem reescrever o sistema do zero — foi cumprido:** o comportamento externo permanece o mesmo, mas o código interno passou a comunicar suas intenções com clareza, validar suas próprias entradas e organizar-se em camadas com responsabilidades distintas.